
OBJECT-ORIENTED PROGRAMMING WITH VB .NET

Topics Covered:

- Classes & Objects
 - Encapsulation
 - Inheritance
 - Polymorphism
- Abstraction & Interfaces
- Constructors & Destructors
 - Properties & Methods
- Exception Handling in OOP

Table of Contents

Table of Contents.....	2
Module 1: Introduction to Object-Oriented Programming.....	4
1.1 What is OOP?	4
1.2 Why Use OOP?	4
1.3 The Four Pillars of OOP	4
1.4 OOP vs Procedural Programming.....	4
Module 2: Classes and Objects.....	6
2.1 Defining a Class in VB.NET	6
2.2 Creating Objects (Instances)	6
2.3 Practical Example: A Student Class.....	6
2.4 Access Modifiers	7
Module 3: Encapsulation.....	8
3.1 What is Encapsulation?	8
3.2 Properties in VB.NET.....	8
3.3 Auto-Implemented Properties	9
3.4 Benefits of Encapsulation	9
Module 4: Constructors and Destructors	10
4.1 Constructors	10
4.2 Default Constructor.....	10
4.3 Parameterized Constructor	10
4.4 Constructor Overloading.....	10
4.5 The Me Keyword	11
4.6 Destructors (Finalize).....	11
Module 5: Inheritance.....	12
5.1 What is Inheritance?	12
5.2 Basic Inheritance Example	12
5.3 The MyBase Keyword.....	12
5.4 Overriding Methods	13
5.5 NotInheritable and MustInherit.....	13
Module 6: Polymorphism.....	15
6.1 What is Polymorphism?	15
6.2 Compile-Time Polymorphism: Method Overloading	15
6.3 Runtime Polymorphism: Method Overriding.....	15
6.4 Interfaces for Polymorphism	16
Module 7: Abstraction and Interfaces	17

7.1 What is Abstraction?	17
7.2 Abstract Classes (MustInherit)	17
7.3 Interfaces	18
7.4 Abstract Class vs Interface	18
Module 8: Exception Handling in OOP	20
8.1 What is Exception Handling?	20
8.2 Try-Catch-Finally	20
8.3 Creating Custom Exceptions	20
Module 9: Working with Collections in OOP	22
9.1 List(Of T) – Generic Collection	22
9.2 Dictionary(Of TKey, TValue)	22
Module 10: Capstone Project – Library Management System	23
10.1 Project Overview	23
10.2 Full Code Implementation	23
Module 11: OOP Best Practices and Design Guidelines	27
11.1 SOLID Principles	27
11.2 Naming Conventions in VB.NET	27
11.3 Common OOP Mistakes to Avoid	27
Module 12: Practice Exercises	28
Exercise 1 – Basic Class Design	28
Exercise 2 – Inheritance	28
Exercise 3 – Interface Implementation	28
Exercise 4 – Exception Handling	28
Exercise 5 – Capstone Mini-Project	28
Assessment Checklist	Error! Bookmark not defined.

Module 1: Introduction to Object-Oriented Programming

1.1 What is OOP?

Object-Oriented Programming (OOP) is a programming paradigm that organizes software design around data, or objects, rather than functions and logic. An object can be defined as a data field that has unique attributes and behavior. OOP focuses on the objects that developers want to manipulate rather than the logic required to manipulate them.

VB.NET is a fully object-oriented language built on the .NET platform. It supports all four main principles of OOP: Encapsulation, Inheritance, Polymorphism, and Abstraction.

1.2 Why Use OOP?

- **Modularity:** Code is organized into discrete objects that can be worked on independently.
- **Reusability:** Objects and classes can be reused across programs.
- **Scalability:** OOP makes it easier to upgrade and scale a program.
- **Maintainability:** Isolated units of code are easier to debug and maintain.
- **Security:** Encapsulation hides data from outside access.

1.3 The Four Pillars of OOP

Pillar	Description
Encapsulation	Bundling data and methods that operate on that data within one unit (class), hiding internal details.
Inheritance	A new class (child) can inherit properties and methods from an existing class (parent).
Polymorphism	Objects of different classes can be treated as objects of a common base class; methods can behave differently.
Abstraction	Hiding complex implementation details while exposing only the necessary parts through interfaces or abstract classes.

1.4 OOP vs Procedural Programming

Procedural Programming	Object-Oriented Programming
Organized around procedures/functions	Organized around objects and classes
Data is passed between functions	Data and behavior are bundled together
Harder to maintain in large programs	Easier to maintain and extend
Top-down approach	Bottom-up approach
Example: Classic VB6, C	Example: VB.NET, C#, Java

Module 2: Classes and Objects

2.1 Defining a Class in VB.NET

A class is a blueprint or template for creating objects. It defines the properties (data) and methods (behaviour) that objects of that class will have. Use the Class keyword in VB.NET to define a class.

Syntax:

```
Public Class ClassName
    ' Fields (variables)
    ' Properties
    ' Methods
    ' Constructors
End Class
```

2.2 Creating Objects (Instances)

An object is a specific instance of a class. You create objects using the New keyword.

```
Dim myObject As New ClassName()

' Or in two steps:
Dim myObject As ClassName
myObject = New ClassName()
```

2.3 Practical Example: A Student Class

```
Public Class Student
    ' Fields
    Private _name As String
    Private _age As Integer
    Private _studentID As String

    ' Properties
    Public Property Name As String
        Get
            Return _name
        End Get
        Set(value As String)
            _name = value
        End Set
    End Property

    Public Property Age As Integer
        Get
            Return _age
        End Get
        Set(value As Integer)
            If value > 0 Then
                _age = value
            End If
        End Set
    End Property

    ' Method
    Public Sub DisplayInfo()
```

```
        Console.WriteLine($"Name: {_name}, Age: {_age}, ID: {_studentID}")
    End Sub
End Class

' --- Using the Student class ---
Dim s1 As New Student()
s1.Name = "Alice"
s1.Age = 21
s1.DisplayInfo()
```

2.4 Access Modifiers

Modifier	Description
Public	Accessible from anywhere in the program.
Private	Accessible only within the same class.
Protected	Accessible within the class and all derived (child) classes.
Friend	Accessible within the same assembly (project).
Protected Friend	Accessible within the class, derived classes, and same assembly.
Note: Best practice: Always make fields Private and expose them through Public Properties to enforce encapsulation.	

Module 3: Encapsulation

3.1 What is Encapsulation?

Encapsulation is the process of wrapping data (fields) and the methods that operate on the data into a single unit called a class, and restricting direct access to some of the object's components. This is achieved by making fields Private and providing Public Properties or Methods to access and modify them.

3.2 Properties in VB.NET

Properties provide a way to read and write private fields in a controlled manner. They have Get and Set accessors.

```
Public Class BankAccount
    Private _balance As Decimal
    Private _accountNumber As String

    ' Read-only property
    Public ReadOnly Property AccountNumber As String
        Get
            Return _accountNumber
        End Get
    End Property

    ' Property with validation
    Public Property Balance As Decimal
        Get
            Return _balance
        End Get
        Set(value As Decimal)
            If value >= 0 Then
                _balance = value
            Else
                Throw New ArgumentException("Balance cannot be negative")
            End If
        End Set
    End Property

    ' Auto-implemented property (shorthand)
    Public Property OwnerName As String

    ' Method to deposit
    Public Sub Deposit(amount As Decimal)
        If amount > 0 Then
            _balance += amount
            Console.WriteLine($"Deposited: {amount:C}. New Balance: {_balance:C}")
        End If
    End Sub

    ' Method to withdraw
    Public Function Withdraw(amount As Decimal) As Boolean
        If amount > 0 AndAlso amount <= _balance Then
            _balance -= amount
            Return True
        End If
        Return False
    End Function
End Class
```

```
End Function  
End Class
```

3.3 Auto-Implemented Properties

VB.NET allows you to define simple properties without explicit Get/Set blocks. The compiler automatically creates a private backing field.

```
Public Class Person  
    ' Auto-implemented properties  
    Public Property FirstName As String  
    Public Property LastName As String  
    Public Property DateOfBirth As Date  
  
    ' Read-only computed property (no setter)  
    Public ReadOnly Property FullName As String  
        Get  
            Return $"{FirstName} {LastName}"  
        End Get  
    End Property  
End Class
```

3.4 Benefits of Encapsulation

- Data hiding: Internal data is protected from unintended external access.
- Validation: Setters can validate data before assigning it.
- Flexibility: Internal implementation can change without affecting external code.
- Maintainability: Centralised control over data access.

Module 4: Constructors and Destructors

4.1 Constructors

A constructor is a special method that is automatically called when an object is created (instantiated). In VB.NET, constructors are defined using the Sub New() method. Constructors are used to initialize the object's properties and fields.

4.2 Default Constructor

```
Public Class Car
    Public Property Make As String
    Public Property Model As String
    Public Property Year As Integer

    ' Default constructor (no parameters)
    Public Sub New()
        Make = "Unknown"
        Model = "Unknown"
        Year = 2000
    End Sub
End Class

' Usage
Dim myCar As New Car() ' Calls the default constructor
Console.WriteLine(myCar.Make) ' Output: Unknown
```

4.3 Parameterized Constructor

```
Public Class Car
    Public Property Make As String
    Public Property Model As String
    Public Property Year As Integer

    ' Parameterized constructor
    Public Sub New(make As String, model As String, year As Integer)
        Me.Make = make
        Me.Model = model
        Me.Year = year
    End Sub
End Class

' Usage
Dim car1 As New Car("Toyota", "Corolla", 2022)
Dim car2 As New Car("BMW", "X5", 2023)
Console.WriteLine($"{car1.Make} {car1.Model} ({car1.Year})")
```

4.4 Constructor Overloading

VB.NET supports multiple constructors with different parameters (overloading).

```
Public Class Employee
    Public Property Name As String
    Public Property Salary As Decimal
    Public Property Dept As String

    ' Constructor 1: Name only
```

```

Public Sub New(name As String)
    Me.Name = name
    Me.Salary = 0
    Me.Dept = "General"
End Sub

' Constructor 2: Name and Salary
Public Sub New(name As String, salary As Decimal)
    Me.Name = name
    Me.Salary = salary
    Me.Dept = "General"
End Sub

' Constructor 3: All parameters
Public Sub New(name As String, salary As Decimal, dept As String)
    Me.Name = name
    Me.Salary = salary
    Me.Dept = dept
End Sub
End Class

```

4.5 The Me Keyword

The Me keyword refers to the current instance of the class. It is used to distinguish between class fields/properties and constructor/method parameters that have the same name.

4.6 Destructors (Finalize)

A destructor (Finalize method) is called by the garbage collector before an object is destroyed. In VB.NET, this is done by overriding the Finalize() method. However, for resource cleanup, implementing IDisposable is generally preferred.

```

Public Class ResourceManager
    Private _resource As String

    Public Sub New(name As String)
        _resource = name
        Console.WriteLine($"Resource '{_resource}' acquired.")
    End Sub

    Protected Overrides Sub Finalize()
        Console.WriteLine($"Resource '{_resource}' released.")
        MyBase.Finalize()
    End Sub
End Class

```

Module 5: Inheritance

5.1 What is Inheritance?

Inheritance is a mechanism where a new class (derived/child class) acquires the properties and methods of an existing class (base/parent class). It promotes code reuse and establishes a parent-child relationship between classes. In VB.NET, use the Inherits keyword.

5.2 Basic Inheritance Example

```
' --- Base (Parent) Class ---
Public Class Animal
    Public Property Name As String
    Public Property Sound As String

    Public Sub New(name As String, sound As String)
        Me.Name = name
        Me.Sound = sound
    End Sub

    Public Sub MakeSound()
        Console.WriteLine($"{Name} says: {Sound}")
    End Sub

    Public Sub Breathe()
        Console.WriteLine($"{Name} is breathing.")
    End Sub
End Class

' --- Derived (Child) Class ---
Public Class Dog
    Inherits Animal

    Public Property Breed As String

    Public Sub New(name As String, breed As String)
        MyBase.New(name, "Woof") ' Call parent constructor
        Me.Breed = breed
    End Sub

    Public Sub Fetch()
        Console.WriteLine($"{Name} fetches the ball!")
    End Sub
End Class

' --- Usage ---
Dim d As New Dog("Rex", "Labrador")
d.MakeSound() ' Inherited from Animal
d.Breathe()   ' Inherited from Animal
d.Fetch()     ' Own method
```

5.3 The MyBase Keyword

MyBase is used to call members (methods and constructors) of the parent class from within a derived class. It is especially useful when the derived class overrides a method but still needs to call the parent's implementation.

5.4 Overriding Methods

To allow a method to be overridden in derived classes, mark it with `Overridable` in the base class and use `Overrides` in the derived class.

```
Public Class Shape
    Public Property Color As String = "White"

    Public Overridable Function CalculateArea() As Double
        Return 0
    End Function

    Public Overridable Sub Describe()
        Console.WriteLine($"I am a {Me.GetType().Name} colored {Color}.")
    End Sub
End Class

Public Class Circle
    Inherits Shape
    Public Property Radius As Double

    Public Sub New(radius As Double, color As String)
        Me.Radius = radius
        Me.Color = color
    End Sub

    Public Overrides Function CalculateArea() As Double
        Return Math.PI * Radius * Radius
    End Function
End Class

Public Class Rectangle
    Inherits Shape
    Public Property Width As Double
    Public Property Height As Double

    Public Sub New(w As Double, h As Double, color As String)
        Me.Width = w
        Me.Height = h
        Me.Color = color
    End Sub

    Public Overrides Function CalculateArea() As Double
        Return Width * Height
    End Function
End Class
```

5.5 NotInheritable and MustInherit

Keyword	Meaning
MustInherit	Makes a class abstract — cannot be instantiated directly; must be inherited.

NotInheritable	Seals a class — prevents it from being inherited (like 'sealed' in C#).
Overridable	Marks a method as virtual — can be overridden in a derived class.
NotOverridable	Prevents a method from being further overridden in subclasses.
MustOverride	Marks a method as abstract — must be overridden in derived classes.

Module 6: Polymorphism

6.1 What is Polymorphism?

Polymorphism (Greek for 'many forms') is the ability of different objects to respond to the same method call in different ways. In OOP, it allows a single interface to represent different underlying forms (data types or classes). VB.NET supports two types of polymorphism: Compile-time (Overloading) and Runtime (Overriding).

6.2 Compile-Time Polymorphism: Method Overloading

Method overloading allows multiple methods with the same name but different parameters in the same class.

```
Public Class Calculator
    ' Overloaded Add methods
    Public Function Add(a As Integer, b As Integer) As Integer
        Return a + b
    End Function

    Public Function Add(a As Double, b As Double) As Double
        Return a + b
    End Function

    Public Function Add(a As Integer, b As Integer, c As Integer) As Integer
        Return a + b + c
    End Function

    Public Function Add(a As String, b As String) As String
        Return a & b ' String concatenation
    End Function
End Class

Dim calc As New Calculator()
Console.WriteLine(calc.Add(3, 4))           ' 7
Console.WriteLine(calc.Add(3.5, 4.5))       ' 8.0
Console.WriteLine(calc.Add(1, 2, 3))        ' 6
Console.WriteLine(calc.Add("Hello ", "World")) ' Hello World
```

6.3 Runtime Polymorphism: Method Overriding

Runtime polymorphism is achieved through method overriding. A base class reference variable can hold a derived class object, and the appropriate method is called at runtime.

```
' Using the Shape hierarchy from Module 5
Dim shapes As New List(Of Shape)
shapes.Add(New Circle(5.0, "Red"))
shapes.Add(New Rectangle(4.0, 6.0, "Blue"))
shapes.Add(New Circle(3.0, "Green"))

' Polymorphic method call - each shape calculates its own area
For Each shape As Shape In shapes
    Console.WriteLine($"{shape.GetType().Name}: Area = {shape.CalculateArea():F2}")
Next

' Output:
```

```
' Circle: Area = 78.54
' Rectangle: Area = 24.00
' Circle: Area = 28.27
```

6.4 Interfaces for Polymorphism

Interfaces define a contract that classes must implement, enabling polymorphism across unrelated classes.

```
Public Interface IPayable
    Function CalculatePay() As Decimal
    Sub PrintPaySlip()
End Interface

Public Class SalariedEmployee
    Implements IPayable
    Public Property Name As String
    Public Property MonthlySalary As Decimal

    Public Function CalculatePay() As Decimal Implements IPayable.CalculatePay
        Return MonthlySalary
    End Function

    Public Sub PrintPaySlip() Implements IPayable.PrintPaySlip
        Console.WriteLine($"[Salaried] {Name}: {CalculatePay():C}/month")
    End Sub
End Class

Public Class HourlyEmployee
    Implements IPayable
    Public Property Name As String
    Public Property HourlyRate As Decimal
    Public Property HoursWorked As Double

    Public Function CalculatePay() As Decimal Implements IPayable.CalculatePay
        Return CDec(HourlyRate * HoursWorked)
    End Function

    Public Sub PrintPaySlip() Implements IPayable.PrintPaySlip
        Console.WriteLine($"[Hourly] {Name}: {CalculatePay():C} this period")
    End Sub
End Class

' Polymorphic usage
Dim payroll As New List(Of IPayable)
payroll.Add(New SalariedEmployee With {.Name="Alice", .MonthlySalary=5000})
payroll.Add(New HourlyEmployee With {.Name="Bob", .HourlyRate=25,
    .HoursWorked=160})

For Each emp As IPayable In payroll
    emp.PrintPaySlip()
Next
```

Module 7: Abstraction and Interfaces

7.1 What is Abstraction?

Abstraction is the process of hiding complex implementation details and showing only the essential features of an object to the user. In VB.NET, abstraction is achieved through Abstract Classes (using MustInherit) and Interfaces.

7.2 Abstract Classes (MustInherit)

An abstract class cannot be instantiated. It serves as a base template for derived classes. Abstract methods (MustOverride) must be implemented in every derived class.

```
Public MustInherit Class Vehicle
    ' Common properties
    Public Property Brand As String
    Public Property Speed As Integer

    Public Sub New(brand As String)
        Me.Brand = brand
    End Sub

    ' Concrete method (shared by all vehicles)
    Public Sub StartEngine()
        Console.WriteLine($"{Brand} engine started.")
    End Sub

    ' Abstract methods (MUST be overridden in subclasses)
    Public MustOverride Sub Refuel()
    Public MustOverride Function MaxRange() As Integer
End Class

Public Class ElectricCar
    Inherits Vehicle

    Public Property BatteryKWh As Double

    Public Sub New(brand As String, battery As Double)
        MyBase.New(brand)
        BatteryKWh = battery
    End Sub

    Public Overrides Sub Refuel()
        Console.WriteLine($"{Brand} is charging its battery ({BatteryKWh} kWh).")
    End Sub

    Public Overrides Function MaxRange() As Integer
        Return CInt(BatteryKWh * 6) ' Approx 6 km per kWh
    End Function
End Class

Public Class PetrolCar
    Inherits Vehicle

    Public Property TankLitres As Double
```

```

Public Sub New(brand As String, tank As Double)
    MyBase.New(brand)
    TankLitres = tank
End Sub

Public Overrides Sub Refuel()
    Console.WriteLine($"{Brand} is filling up ({TankLitres}L tank).")
End Sub

Public Overrides Function MaxRange() As Integer
    Return CInt(TankLitres * 12) ' Approx 12 km per litre
End Function
End Class

```

7.3 Interfaces

An interface is a pure contract — it defines method signatures, properties, and events that a class must implement, but provides no implementation. A class can implement multiple interfaces.

```

Public Interface IFlyable
    Sub TakeOff()
    Sub Land()
    ReadOnly Property MaxAltitude As Integer
End Interface

Public Interface ISwimmable
    Sub Dive()
    Sub Surface()
End Interface

' A class can implement multiple interfaces
Public Class Duck
    Implements IFlyable, ISwimmable

    Public Sub TakeOff() Implements IFlyable.TakeOff
        Console.WriteLine("Duck takes off!")
    End Sub

    Public Sub Land() Implements IFlyable.Land
        Console.WriteLine("Duck lands.")
    End Sub

    Public ReadOnly Property MaxAltitude As Integer Implements IFlyable.MaxAltitude
        Get
            Return 1000
        End Get
    End Property

    Public Sub Dive() Implements ISwimmable.Dive
        Console.WriteLine("Duck dives underwater.")
    End Sub

    Public Sub Surface() Implements ISwimmable.Surface
        Console.WriteLine("Duck surfaces.")
    End Sub
End Class

```

7.4 Abstract Class vs Interface

Abstract Class (MustInherit)	Interface
Can have concrete and abstract methods	Only method signatures (no implementation)
Can have fields, constructors, and state	Cannot have fields or constructors
A class can inherit only ONE abstract class	A class can implement MULTIPLE interfaces
Use when classes share common base behavior	Use to define a capability/contract across unrelated classes

Module 8: Exception Handling in OOP

8.1 What is Exception Handling?

Exception handling is a mechanism to handle runtime errors gracefully, preventing the program from crashing. In VB.NET, exceptions are objects. Good OOP design includes proper exception handling to make classes robust and reliable.

8.2 Try-Catch-Finally

```
Public Sub DivideNumbers(a As Integer, b As Integer)
    Try
        Dim result As Double = a / b
        Console.WriteLine($"{a} / {b} = {result}")
    Catch ex As DivideByZeroException
        Console.WriteLine("Error: Cannot divide by zero.")
    Catch ex As Exception
        Console.WriteLine($"Unexpected error: {ex.Message}")
    Finally
        Console.WriteLine("DivideNumbers completed.")
    End Try
End Sub
```

8.3 Creating Custom Exceptions

You can create custom exception classes that extend the built-in Exception class, allowing you to define meaningful error types specific to your application.

```
' Custom exception class
Public Class InsufficientFundsException
    Inherits Exception

    Public Property Amount As Decimal

    Public Sub New(amount As Decimal)
        MyBase.New($"Insufficient funds. Short by: {amount:C}")
        Me.Amount = amount
    End Sub
End Class

' Using the custom exception in a class
Public Class BankAccount
    Private _balance As Decimal

    Public Sub New(initialBalance As Decimal)
        _balance = initialBalance
    End Sub

    Public Sub Withdraw(amount As Decimal)
        If amount > _balance Then
            Throw New InsufficientFundsException(amount - _balance)
        End If
        _balance -= amount
        Console.WriteLine($"Withdrew {amount:C}. Balance: {_balance:C}")
    End Sub
End Class
```

```
' Handling the custom exception
Dim account As New BankAccount(500)
Try
    account.Withdraw(800)
Catch ex As InsufficientFundsException
    Console.WriteLine($"Cannot withdraw: {ex.Message}")
End Try
```

Module 9: Working with Collections in OOP

9.1 List(Of T) – Generic Collection

The generic List(Of T) class is commonly used to store collections of objects. It is type-safe and offers powerful methods for searching, sorting, and manipulating data.

```
Public Class StudentManager
    Private _students As New List(Of Student) ()

    Public Sub AddStudent(s As Student)
        _students.Add(s)
        Console.WriteLine($"Added: {s.Name}")
    End Sub

    Public Sub RemoveStudent(name As String)
        Dim target = _students.Find(Function(s) s.Name = name)
        If target IsNot Nothing Then
            _students.Remove(target)
            Console.WriteLine($"Removed: {name}")
        End If
    End Sub

    Public Sub DisplayAll()
        For Each s As Student In _students
            s.DisplayInfo()
        Next
    End Sub

    Public Function SearchByName(name As String) As Student
        Return _students.Find(Function(s) s.Name.ToLower() = name.ToLower())
    End Function
End Class
```

9.2 Dictionary(Of TKey, TValue)

```
' Store students by their ID
Dim studentDB As New Dictionary(Of String, Student) ()

studentDB("S001") = New Student With {.Name = "Alice", .Age = 20}
studentDB("S002") = New Student With {.Name = "Bob", .Age = 22}

' Retrieve a student
If studentDB.ContainsKey("S001") Then
    Console.WriteLine(studentDB("S001").Name)
End If

' Iterate
For Each kvp As KeyValuePair(Of String, Student) In studentDB
    Console.WriteLine($"ID: {kvp.Key} - Name: {kvp.Value.Name}")
Next
```

Module 10: Capstone Project – Library Management System

This capstone project integrates all OOP concepts covered in this module. Students will build a simple Library Management System that demonstrates classes, inheritance, polymorphism, encapsulation, interfaces, and exception handling.

10.1 Project Overview

Class / Interface	Purpose
ILoanable (Interface)	Defines CheckOut() and ReturnItem() contract
LibraryItem (Abstract)	Base class for all library items
Book (Derived)	Represents a physical book
EBook (Derived)	Represents a digital book
Member (Class)	Library member with name and borrowed items
Library (Class)	Manages items and members, handles loans
ItemNotFoundException	Custom exception when item is not found

10.2 Full Code Implementation

```
' =====  
' LIBRARY MANAGEMENT SYSTEM - VB.NET OOP Capstone Project  
' =====  
  
' --- Interface ---  
Public Interface ILoanable  
    Sub CheckOut(member As Member)  
    Sub ReturnItem()  
    ReadOnly Property IsAvailable As Boolean  
End Interface  
  
' --- Custom Exception ---  
Public Class ItemNotFoundException  
    Inherits Exception  
    Public Sub New(title As String)  
        MyBase.New($"Item '{title}' was not found in the library.")  
    End Sub  
End Class  
  
' --- Abstract Base Class ---  
Public MustInherit Class LibraryItem  
    Implements ILoanable  
  
    Public Property ItemID As String  
    Public Property Title As String  
    Public Property Author As String  
    Protected _isAvailable As Boolean = True  
    Protected _currentMember As Member
```

```

    Public Sub New(id As String, title As String, author As String)
        Me.ItemID = id
        Me.Title = title
        Me.Author = author
    End Sub

    Public ReadOnly Property IsAvailable As Boolean _
        Implements ILoanable.IsAvailable
        Get
            Return _isAvailable
        End Get
    End Property

    Public Sub CheckOut(member As Member) Implements ILoanable.CheckOut
        If Not _isAvailable Then
            Throw New Exception($"'{Title}' is already checked out.")
        End If
        _isAvailable = False
        _currentMember = member
        Console.WriteLine($"'{Title}' checked out to {member.Name}.")
    End Sub

    Public Sub ReturnItem() Implements ILoanable.ReturnItem
        _isAvailable = True
        _currentMember = Nothing
        Console.WriteLine($"'{Title}' returned to library.")
    End Sub

    Public MustOverride Sub DisplayDetails()
End Class

' --- Concrete Classes ---
Public Class Book
    Inherits LibraryItem
    Public Property Pages As Integer

    Public Sub New(id As String, title As String, author As String, pages As Integer)
        MyBase.New(id, title, author)
        Me.Pages = pages
    End Sub

    Public Overrides Sub DisplayDetails()
        Dim status = If(_isAvailable, "Available", "Checked Out")
        Console.WriteLine($"[BOOK] {ItemID}: '{Title}' by {Author} | {Pages} pages | {status}")
    End Sub
End Class

Public Class EBook
    Inherits LibraryItem
    Public Property FileSizeMB As Double
    Public Property Format As String

    Public Sub New(id As String, title As String, author As String, _
        size As Double, format As String)
        MyBase.New(id, title, author)
        FileSizeMB = size
        Me.Format = format
    End Sub

```

```

        Public Overrides Sub DisplayDetails()
            Dim status = If(_isAvailable, "Available", "Checked Out")
            Console.WriteLine($"[EBOOK] {ItemID}: '{Title}' by {Author} | {FileSizeMB}MB {Format} | {status}")
        End Sub
    End Class

' --- Member Class ---
Public Class Member
    Public Property MemberID As String
    Public Property Name As String
    Private _borrowedItems As New List(Of LibraryItem)()

    Public Sub New(id As String, name As String)
        MemberID = id
        Me.Name = name
    End Sub

    Public Sub BorrowItem(item As LibraryItem)
        _borrowedItems.Add(item)
    End Sub

    Public Sub DisplayBorrowedItems()
        Console.WriteLine($"{Name}'s borrowed items:")
        If _borrowedItems.Count = 0 Then
            Console.WriteLine(" None.")
        Else
            For Each item In _borrowedItems
                Console.WriteLine($" - {item.Title}")
            Next
        End If
    End Sub
End Class

' --- Library Class ---
Public Class Library
    Private _items As New List(Of LibraryItem)()
    Private _members As New List(Of Member)()

    Public Sub AddItem(item As LibraryItem)
        _items.Add(item)
    End Sub

    Public Sub RegisterMember(m As Member)
        _members.Add(m)
    End Sub

    Public Sub LoanItem(title As String, memberID As String)
        Try
            Dim item = _items.Find(Function(i) i.Title = title)
            Dim member = _members.Find(Function(m) m.MemberID = memberID)

            If item Is Nothing Then Throw New ItemNotFoundException(title)
            If member Is Nothing Then Throw New Exception($"Member '{memberID}' not found.")

            item.CheckOut(member)
            member.BorrowItem(item)
        Catch ex As ItemNotFoundException
            Console.WriteLine($"Loan Error: {ex.Message}")
        Catch ex As Exception

```

```

        Console.WriteLine($"Loan Error: {ex.Message}")
    End Try
End Sub

Public Sub DisplayCatalogue()
    Console.WriteLine("=== Library Catalogue ===")
    For Each item In _items
        item.DisplayDetails()
    Next
End Sub
End Class

' =====
' MAIN PROGRAM
' =====
Dim lib As New Library()

lib.AddItem(New Book("B001", "Clean Code", "Robert Martin", 464, ))
lib.AddItem(New Book("B002", "The Pragmatic Prog.", "David Thomas", 352, ))
lib.AddItem(New EBook("E001", "VB.NET Deep Dive", "Jane Smith", 12.5, "PDF"))

Dim alice As New Member("M001", "Alice")
Dim bob As New Member("M002", "Bob")
lib.RegisterMember(alice)
lib.RegisterMember(bob)

lib.DisplayCatalogue()
lib.LoanItem("Clean Code", "M001")
lib.LoanItem("Clean Code", "M002") ' Should fail - already loaned
alice.DisplayBorrowedItems()

```

Module 11: OOP Best Practices and Design Guidelines

11.1 SOLID Principles

Principle	Guideline
S – Single Responsibility	A class should have only one reason to change; it should do one thing.
O – Open/Closed	Classes should be open for extension but closed for modification.
L – Liskov Substitution	Derived classes should be substitutable for their base classes.
I – Interface Segregation	Clients should not be forced to depend on interfaces they do not use.
D – Dependency Inversion	Depend on abstractions (interfaces), not concrete implementations.

11.2 Naming Conventions in VB.NET

Element	Convention
Classes	PascalCase: CustomerOrder, StudentRecord
Methods	PascalCase: CalculateTotal(), GetDetails()
Properties	PascalCase: FirstName, AccountBalance
Private Fields	_camelCase: _name, _balance, _studentID
Local Variables	camelCase: totalAmount, firstName
Constants	UPPER_CASE or PascalCase: MAX_RETRIES, Pi
Interfaces	IPrefix + PascalCase: IComparable, IPayable

11.3 Common OOP Mistakes to Avoid

- Overusing inheritance — prefer composition over inheritance when possible.
- Making everything public — always use the most restrictive access modifier.
- Creating 'god classes' that do too many things — apply Single Responsibility.
- Ignoring exception handling inside class methods.
- Not validating input in property setters.
- Creating circular dependencies between classes.
- Using shared (static) state excessively — leads to hard-to-test code.

Module 12: Practice Exercises

Exercise 1 – Basic Class Design

Create a class called Rectangle with private fields for width and height. Add a parameterized constructor, read/write properties with validation (no negative values), and methods for CalculateArea() and CalculatePerimeter(). Instantiate two rectangles and print their areas and perimeters.

Exercise 2 – Inheritance

Design a class hierarchy for a school system. Create a base class Person with Name, Age, and Address. Derive a Student class with StudentID and GradeLevel, and a Teacher class with TeacherID and Subject. Each class should have a DisplayInfo() method overriding the base. Create a list of Person objects containing both students and teachers, and call DisplayInfo() polymorphically.

Exercise 3 – Interface Implementation

Define an interface ICalculatable with methods Add(), Subtract(), Multiply(), and Divide(). Create two classes that implement this interface: SimpleCalculator (basic arithmetic) and ScientificCalculator (which adds Power() and SquareRoot() methods). Demonstrate polymorphism using the interface reference.

Exercise 4 – Exception Handling

Extend the BankAccount class from Module 8. Add a custom NegativeDepositException and a WithdrawalLimitExceededException. Set a maximum withdrawal limit of 10,000 per transaction. Test all exception scenarios and ensure the account balance is never corrupted.

Exercise 5 – Capstone Mini-Project

Build a simple Inventory Management System with the following classes: Product (abstract, with ID, Name, Price), PhysicalProduct (adds weight and stock), DigitalProduct (adds download URL and license key), an IDiscountable interface with CalculateDiscount(), and an Inventory class that manages a list of products and can generate a stock report. Include appropriate exception handling and demonstrate all OOP principles.